

C Programming

Preprocessing, Multisource Files & Dynamic Memory: Ch.16-17

Macros | Header files | malloc/free | Linked lists

What You Will Learn in These Chapters

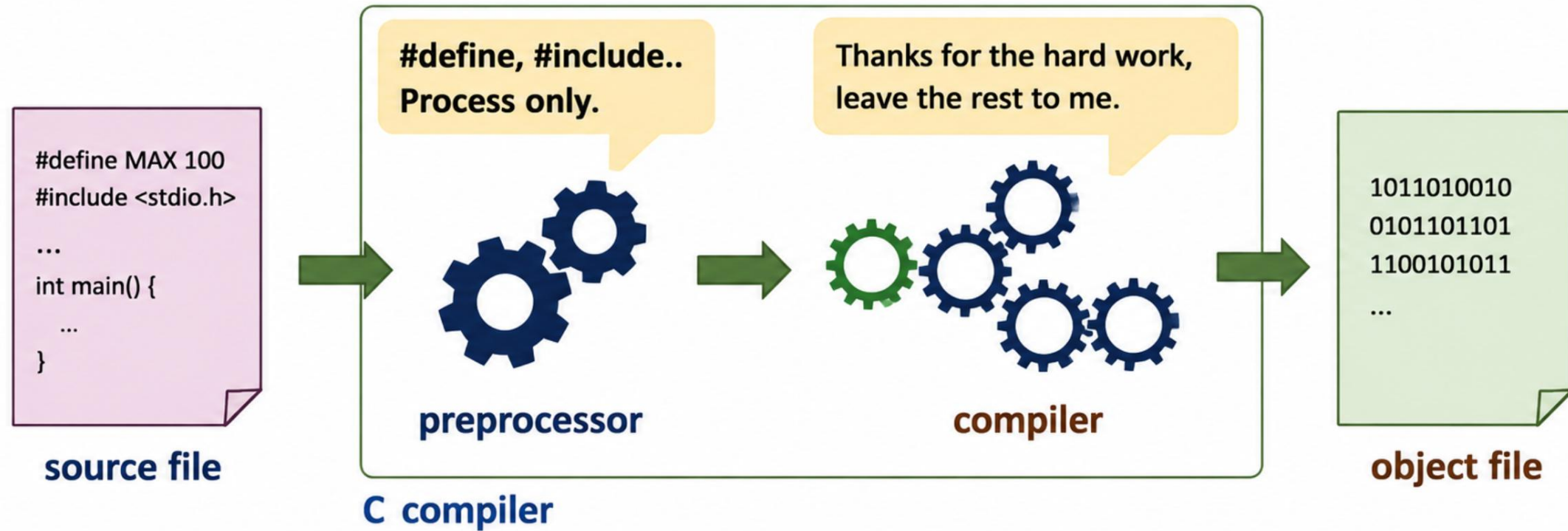
- Preprocessing directives, split compilation, command line parameters, how to debug
- Understanding **dynamic memory** allocation, functions related to it

Ch. 16

Preprocessing and Multisource Files

What Is a Preprocessor?

- **A *preprocessor*** is a part of a compiler that processes source files before compilation.



C Preprocessor Directives

Directive	Meaning	Example	Explanation
#define	Macro definition	#define PI 3.14159	Defines a macro named PI.
#include	Including files	#include <stdio.h>	Includes the standard I/O library.
#undef	Undefine a macro	#undef PI	Removes the definition of PI.
#if	If the condition is true	#if (X > 0)	Begins a conditional block if true.
#else	If the condition is false	#else	Begins the block if the condition is false.
#endif	End of conditional processing	#endif	Ends the conditional block.
#ifdef	If a macro is defined	#ifdef DEBUG	Checks if DEBUG is defined.
#ifndef	If the macro is not defined	#ifndef DEBUG	Checks if DEBUG is not defined.
#line	Print (set line information)	#line 100 "file.c"	Sets the line number to 100 for file.c.
#pragma	Meaning varies by system	#pragma once	Example: include a file only once.

Macro

- A macro is a rule or pattern that specifies how input text should be transformed into replacement text during the preprocessing stage of compilation.

1. What is a Macro?

A macro is defined using the **#define** directive.

Syntax

Simple macro definition

```
#define MAX_SIZE 100
```

Symbol constant

Defined as 100



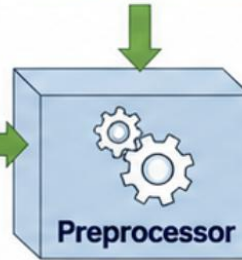
MAX_SIZE is easier to understand than 100.

2. How Macro Works (Preprocessor Substitution)

The preprocessor replaces the macro name with its defined value before compilation.

```
#define MAX_SIZE 100
```

```
...  
while(i<MAX_SIZE)  
{  
    sum += i;  
    i++;  
}  
...
```



```
...  
while(i<100)  
{  
    sum += i;  
    i++;  
}  
...
```

After preprocessing, the compiler sees the code with **100**.

3. Advantages of Macros

- Improves the readability of the program.
- It is easy to change constants.

Before

```
#define MAX_SIZE 100  
for(i=0;i<MAX_SIZE;i++)  
{  
    f += (float) i/MAX_SIZE;  
}
```

After changing one value

```
#define MAX_SIZE 200  
for(i=0;i<MAX_SIZE;i++)  
{  
    f += (float) i/MAX_SIZE;  
}
```

Change only here, everywhere is updated!

Why Use Macros?



Readability:

Meaningful names instead of magic numbers.



Maintainability:

Change a constant in one place.



Consistency:

Same value used across the program.

Before

```
#define MAX_SIZE 100  
for(i=0;i<MAX_SIZE;i++) {  
    f += (float) i/MAX_SIZE;  
}
```

After

```
#define MAX_SIZE 200  
for(i=0;i<MAX_SIZE;i++) {  
    f += (float) i/MAX_SIZE;  
}
```

Change
the constant

Only one change
(100 → 200),
the whole program
uses the **new value**.

Example of a Macro

```
#define PI 3.141592 // pi
#define EOF (-1) // End of file indicator
#define EPS 1.0e-9 // Calculation limit for real numbers
#define DIGITS "0123456789" // Define character constants
#define BRACKET "(){}[]" // Define character constants
```

Using #define to Rename Operators

```
#include <stdio.h>
#define AND &&
#define OR ||
#define IS ==

int search( int list[], int n, int key)
{
    int i = 0;
    while ( i < n AND list[ i ] != key )
        i ++;
    if ( i IS n )
        return -1;
    else
        return i ;
}
```


Preprocessor Directives for Macros

1. Macro Declaration Location (#ifdef)

DEBUG is defined

```
#define DEBUG

int average(int x, int y)
{
#ifdef DEBUG
    printf("x=%d, y=%d\n", x, y);
#endif

    return (x+y)/2;
}
```

Code inside #ifdef DEBUG is included.

DEBUG is not defined

```
int average(int x, int y)
{
#ifdef DEBUG
    printf("x=%d, y=%d\n", x, y);
#endif

    return (x+y)/2;
}
```

Code inside #ifdef DEBUG is not included.

2. #ifndef , #undef

• #ifndef

Includes code only if the macro is not defined.

```
#ifndef LIMIT
#define LIMIT 1000
#endif
```

If LIMIT is not defined

Now define LIMIT

• #undef

Cancels the definition of a macro.

```
#define SIZE 100
...
#undef SIZE
#define SIZE 200
```

Cancels the definition

Directive	Meaning	When It Is True / Action	Example	Result
#ifdef	If the macro is defined	Includes the code between #ifdef and #endif	<pre>#ifdef DEBUG printf("debug info\n"); #endif</pre>	➡ Included if DEBUG is defined.
#ifndef	If the macro is not defined	Includes the code between #ifndef and #endif	<pre>#ifndef LIMIT #define LIMIT 1000 #endif</pre>	➡ Defines LIMIT only if not defined.
#endif	End of conditional block	Marks the end of #if , #ifdef , or #ifndef block	<pre>#ifdef DEBUG ... #endif</pre>	➡ Ends the block.
#undef	Undefine (cancel) a macro	Removes the macro definition	<pre>#undef SIZE</pre>	➡ SIZE is no longer defined.

Conditional Compilation: #ifdef

```
#include <stdio.h>
#define LINUX

int main( void )
{
#ifdef LINUX
    printf ("This is the Linux version.\n");
#else
    printf ("Windows version.\n");
#endif
    return 0;
}
```

Include output statements only when a certain macro (e.g. LINUX or DEBUG) is declared - useful for platform-specific or debug-only code.

Multiple Source Files in C

1. Single Source File

C

big_program.c

```
int main() {  
    ...  
    ...  
}
```

- The file size is too large.
- Difficult to manage and maintain.
- Difficult to reuse source code.

➡ Disadvantage:

As the program grows, it becomes hard to understand and modify.

2. Multiple Source Files

C

main.c

```
int main() {  
    ...  
}
```

C

power.c

```
int power() {  
    ...  
}
```

C

util.c

```
int add() {  
    ...  
}
```

...

- You can collect only related codes into a single source file.
- Easy to manage and maintain.
- Easy to reuse source files in other projects.



Advantage: Improves readability, maintainability, and reusability of the program.

How It Works (Compilation & Linking)

1 Compilation

Each source file (.c) is compiled separately into an object file (.o).

main.c



main.o

power.c



power.o

```
gcc -c main.c  
gcc -c power.c  
...
```

2 Linking

Object files are linked together to create an executable file.



```
gcc main.o power.o -o main
```

3 Executable

The executable file (.exe) is ready to run.



```
./main (Run)
```

Summary of Commands

1 Compile each source file

```
gcc -c main.c  
gcc -c power.c  
...
```



2 Link object files

```
gcc main.o power.o -o main
```



3 Run the program

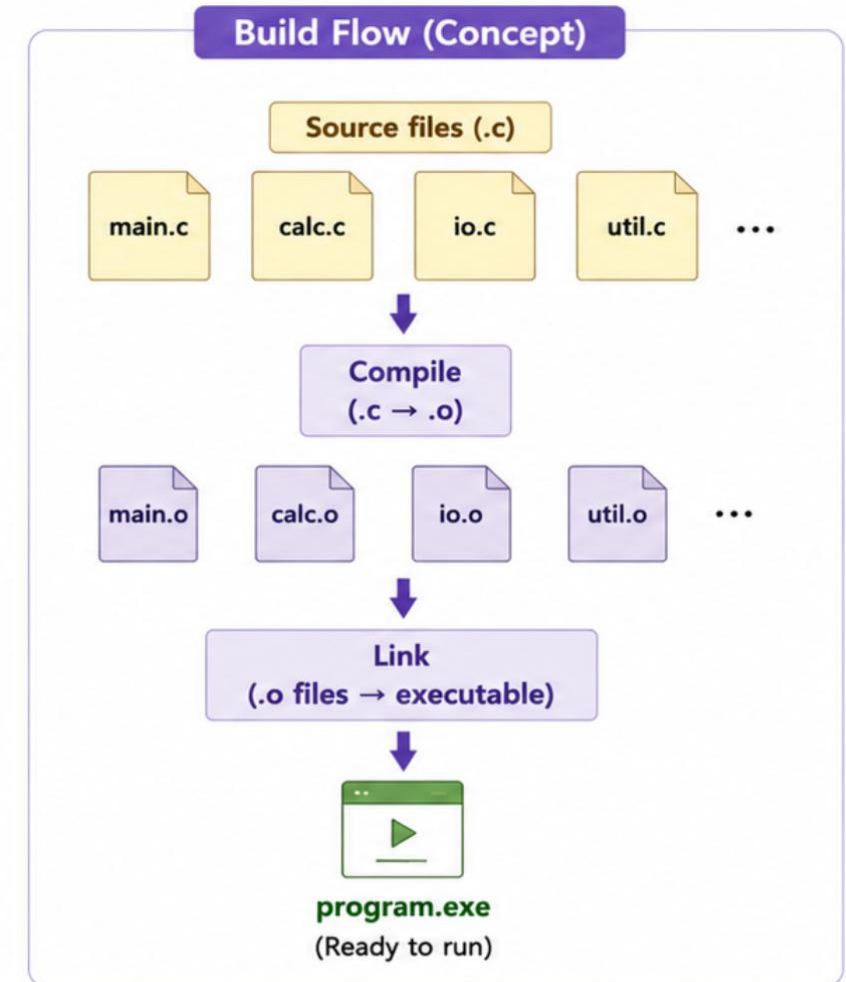
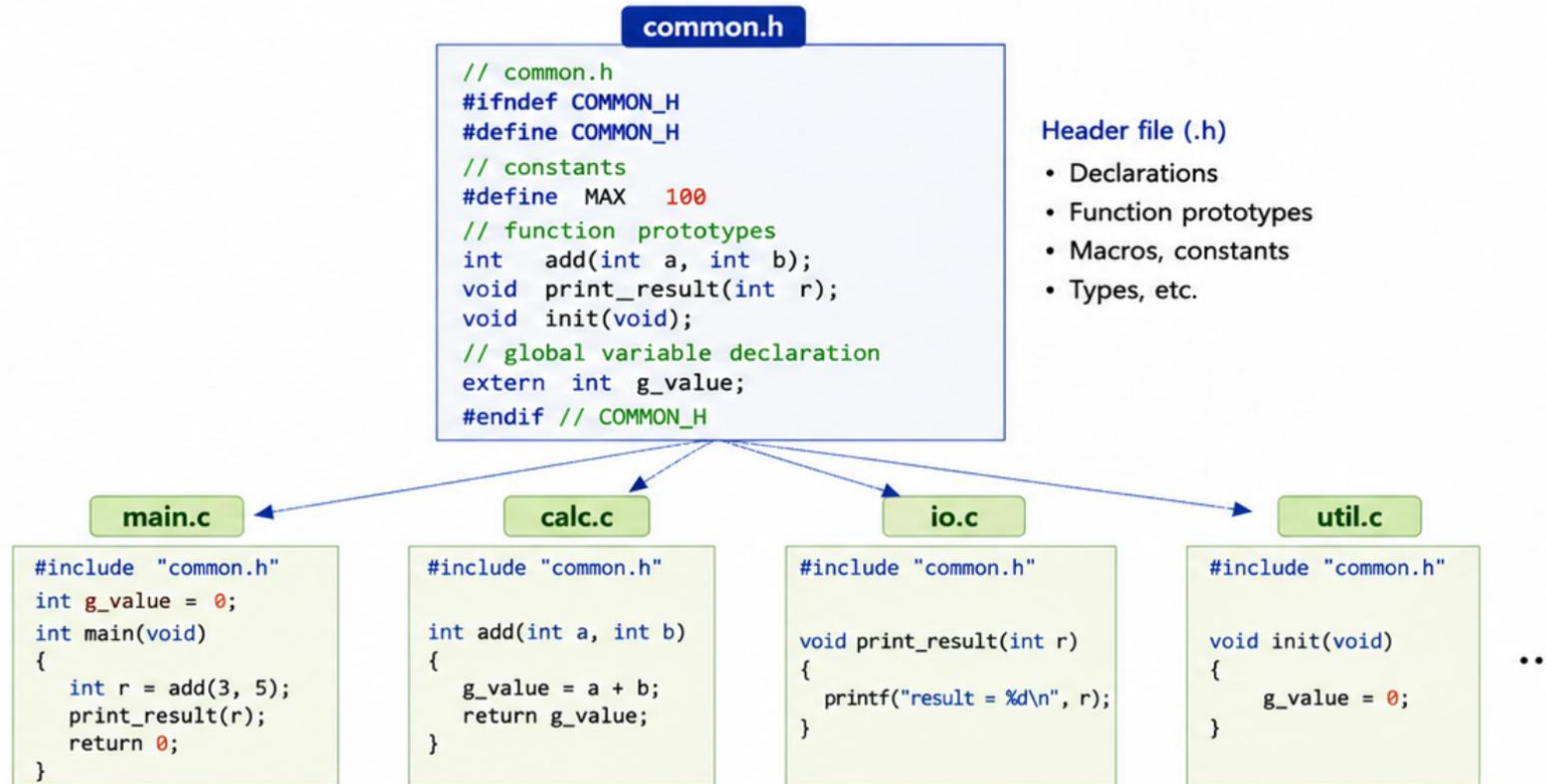
```
./main
```

- .C : Source file (text)
- .O : Object file (machine code)
- .exe : Executable file (ready to run)

Multiple Source Files in C with Header File

- Multiple source files can share one header file.

- Common declarations (functions, variables, macros, constants, types, etc.) are placed in a header file (.h).
- Each source file (.c) includes the header file to use those declarations.



Benefits



Easy to manage and maintain



Easy to reuse in other projects



Changes in one place (header) affect all source files



Improves consistency and reduces errors



Key Point: The header file (.h) contains declarations. Multiple source files (.c) include it and share the same interface.

➔ **Change the header once, and all source files are updated together!**

Example: main.c and power.c

```
// main.c
#include <stdio.h>
#include "power.h"

int main( void )
{
    int x, y;
    printf ( "Enter the value of x: " );
    scanf ( "%d" , &x);
    printf ( "Enter the value of y: " );
    scanf ( "%d" , &y);
    printf ( "%d to the power of %d is %f\n" , x, y, power(x, y));
    return 0;
}

// power.c
#include "power.h"
double power( int x , int y )
{
    double result = 1.0;
    for (int i = 0; i < y ; i++) result *= x ;
    return result;
}
```

Header Files and Header Guards

```
// power.h
#ifndef POWER_H
#define POWER_H

double power( int x , int y ); // Function prototype definition

#endif

// Modern shortcut, same effect:
// #pragma once
```

Lab: header file duplicate blocking - if a header is included twice, you get a compilation error unless you use #ifndef / #define / #endif.

Ch. 17

Dynamic Memory and Linked Lists

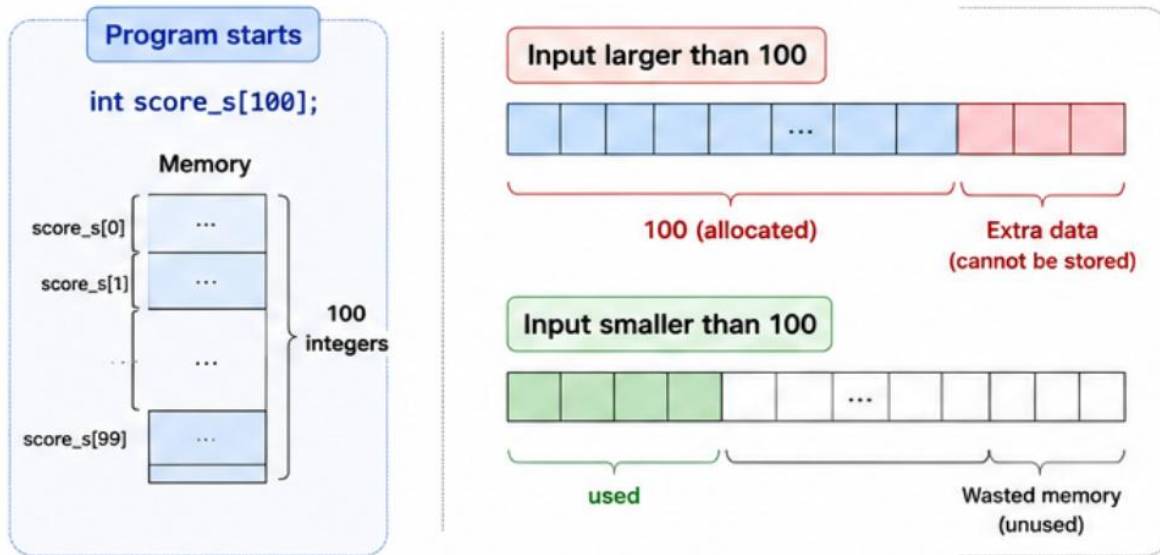
Static vs. Dynamic Memory Allocation

- Static memory allocation: allocating a predetermined amount of memory before the program starts, e.g. `int score_s[100];`
- If an input larger than the initially determined size comes in, it cannot be processed. If a smaller input comes in, the remaining memory space is wasted.
- **Dynamic memory** allocation: dynamically allocating memory during execution, and returning it to the system when finished using it.
- `score = (int *) malloc(100 * sizeof(int));`
 - allocate only as much as you need, and use memory very efficiently.

Static vs. Dynamic Memory Allocation

Static Memory Allocation (정적 메모리 할당)

- Allocating a predetermined amount of memory before the program starts.
- The size of the memory is determined before the program starts.
- (Example) `int score_s[100];`
- If an input larger than the initially determined size comes in, it cannot be processed.
- If a smaller input comes in, the remaining memory space is wasted.

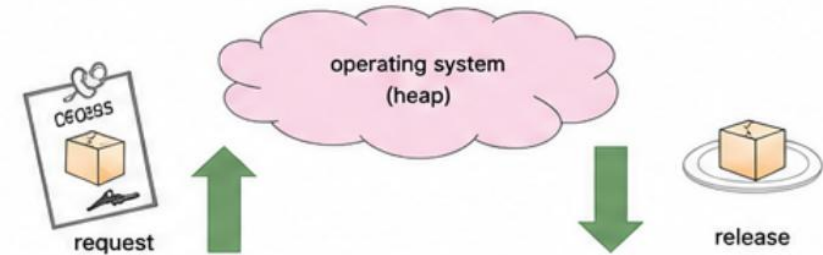


Example

```
int main(void)
{
    int score_s[100]; // memory for 100 integers is allocated
    ...
}
```

Dynamic Memory Allocation (동적 메모리 할당)

- Dynamically allocating memory during execution.
- Return memory to the system when finished using it.
- `score = (int *) malloc(100 * sizeof(int));`
- Allocate only as much as you need and use memory very efficiently.



```
int main(void)
{
    int *score;
    int n;
    ...
    n = get_input_size(); // e.g., read the number of elements
    score = (int *) malloc(n * sizeof(int)); // allocate at runtime
    if (score == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    ... use score ...

    free(score); // return memory when done
    return 0;
}
```

Dynamic Memory Allocation

1. Dynamic Memory Allocation

- Allocates memory at runtime using `malloc()`.

Syntax

Dynamic memory allocation

ex.

```
int *p;
p = (int *)malloc(100 * sizeof(int));
```

Address returned (pointer to allocated memory)

Number of bytes needed (100 integers)

// Allocate 100 integers

p = 0x7ffdf000



100 integers = 100 * 4 bytes = 400 bytes
(assuming int is 4 bytes)

2. malloc() Return Value Must Be Checked

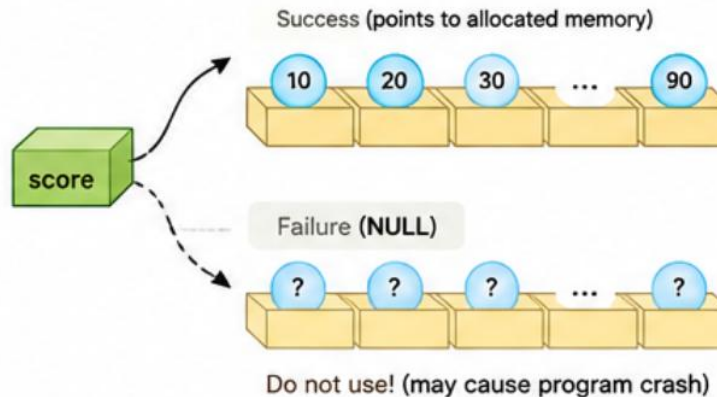
- `malloc()` can fail and return `NULL`.
- Always check the return value before using it.

```
int *score;
score = (int *)malloc(100 * sizeof(int));
```

convert to int pointer

```
if (score == NULL) {
    printf("Memory allocation failed!\n");
    return 1; // or error handling
}
```

Check if memory is allocated correctly



3. Dynamic Memory Release

- Free the allocated memory when it is no longer needed using `free()`.

Syntax

Dynamic memory release

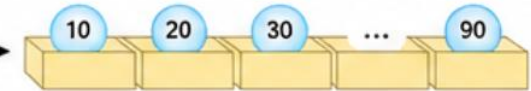
ex.

```
score = (int *)malloc(100 * sizeof(int));
...
free(score);
```

// Returns the allocated memory
// back to the system

Before free()

score



After free()

score

Memory returned to the system (available)

Dynamic Memory Usage: Pointer or Array Style

```
int *score = NULL;
score = (int *)malloc(100*sizeof(int));

// First method: using pointers
*score = 100;
*(score+1) = 200;
*(score+2) = 300;

// Second method: treat dynamic memory like an array
score[0] = 100;
score[1] = 200;
score[2] = 300;
```

Allocating and Freeing Memory

```
#include <stdio.h>
#include <stdlib.h>

int main( void )
{
    int *pi; double *pf; char *pc;

    pi = ( int *)malloc( sizeof ( int ));
    pf = (double *)malloc( sizeof (double));
    pc = (char *)malloc( sizeof (char));
    if (pi == NULL || pf == NULL || pc == NULL ) {
        printf ( "Dynamic memory allocation error\n" );
        exit(1);
    }

    *pi = 100; *pf = 3.14; *pc = 'a';

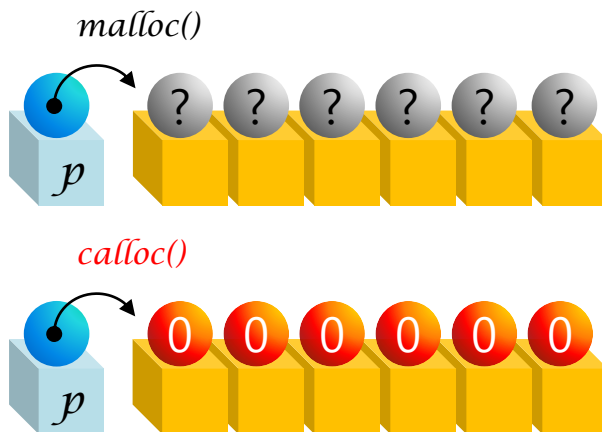
    free(pi); free(pf); free(pc);
    return 0;
}
```

malloc()'s return value must always be checked for NULL.

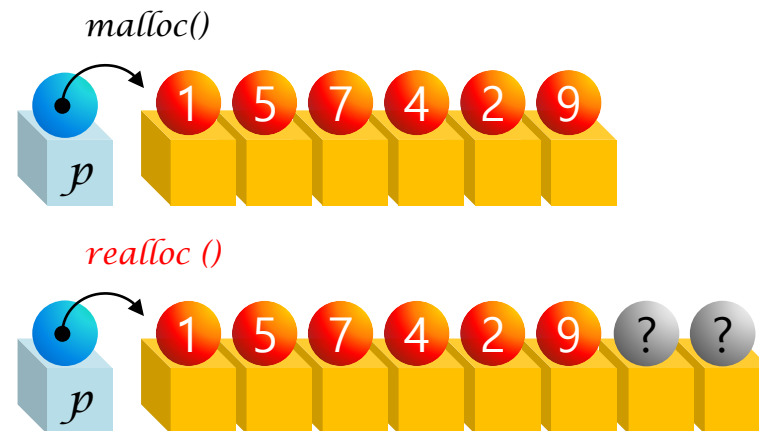
calloc() and realloc()

Function	Purpose
<code>calloc(n, size)</code>	Allocates memory for n items, initialized to 0 (item-by-item)
<code>realloc(ptr, new_size)</code>	Changes the size of a previously allocated memory block, e.g. <code>p = realloc(p, 7 * sizeof(int));</code>

```
int *p;  
p = (int *) calloc (5, sizeof (int));
```



```
int *p ;  
p = (int *) malloc (5 * sizeof (int));  
p = realloc (p, 7 * sizeof (int));
```



Common Mistakes & Tips

- The **preprocessor** handles `#define`, `#ifdef`, and `#include` before compilation.
- **#define** macros are simple text substitution
 - no type checking, and no semicolon at the end.
- Header files + header guards let large programs be split across **multiple source files**.
- **#include <...>** searches system directories;
#include "..." checks the local project folder first.
- **malloc()/calloc()/realloc()/free()** manage memory manually, at run time
 - always check for NULL, and always **free()** what you allocate.
- Always match every **malloc()** with a **free()**
 - forgetting to free memory causes a memory leak.
 - set it to NULL to be safe.

Q & A

Lab & Quiz: right after this - see you in the practice session!